



Software Engineering (sql)

To see UML projects click [here](#).

👉 [UML projects](#)

UML

UML is the acronym of Unified modeling language it was created in 1994 and upgraded until 2017. The software engineers use this model to project software.

Class and object

A program is created modeling a set of classes

A class is defined by a name and a set of properties.

example:

Book
title: String
year: int
author: String

| The level of the classes is called intentional level

an Object is an instance of a Class and is indicated with an univocal identifier.

example:

divina_commedia
title: "divina commedia"
year: 1321
author: "Dante Alighieri"

| The level of the classes is called extensional level

Important cases in UML

| this section is very useful to comprehend how modelling an UML scheme.

- can two identical object co-exist?

The answer is yes.

example:

Book
title:
String
year: int
author:
String

<<instance of book>>

<u>divina_commedia</u>
title: "divina commedia"
year: 1321
author: "Dante Alighieri"

<<instance of book>>

<u>divina_commedia2</u>
title: "divina commedia"
year: 1321
author: "Dante Alighieri"

Identical object can coexist.

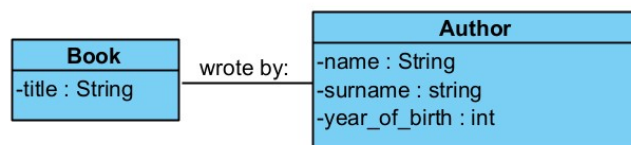
Associations and Links

Two or more classes can have a relationship; we use associations to connect them.

an instance of an association is called link.

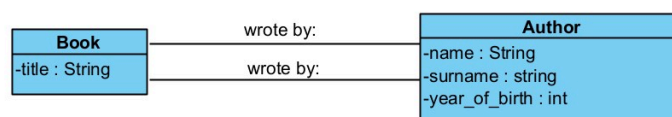
| "Association" in intentional level, "link" in extensional level

example:



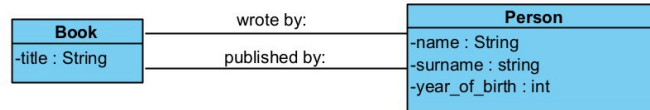
example 2:

UML schema in the next example is not allowed.



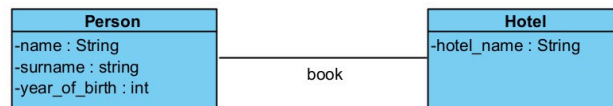
An author can not be author of a book twice, this UML schema doesn't have any sens.

While this is allowed.

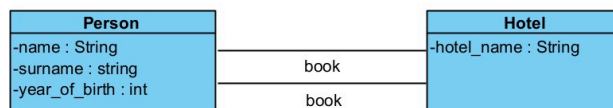


A book has got an author and a publisher, publisher and author is instances of person class.

example 3:

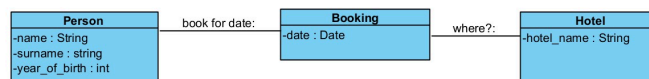


This is an incorrect UML schema because a person can't book the same hotel twice, unless the instance of person is deleted, but this is not efficient.



this is also wrong.

The right version is:

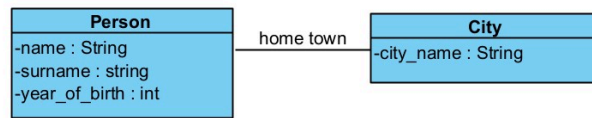


Now we can create different instance of Booking to allow to a person to book the same hotel various times.

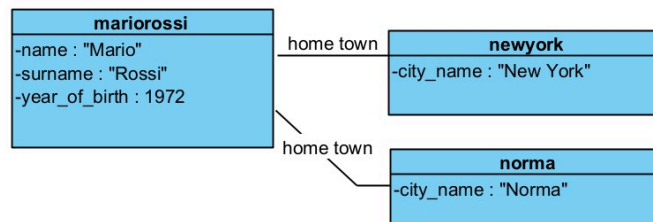
Integrity constraint

Is given this UML schema:

Intentional level:

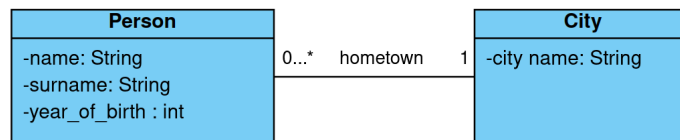


(hypothetical) extensional level:



Utilizing the roles seen, this schema is correct, but sir. Mario Rossi, can't have two different hometowns in real life... This UML schema is not concrete.

To solve this type of problems, we can use the integrity constraint (molt).



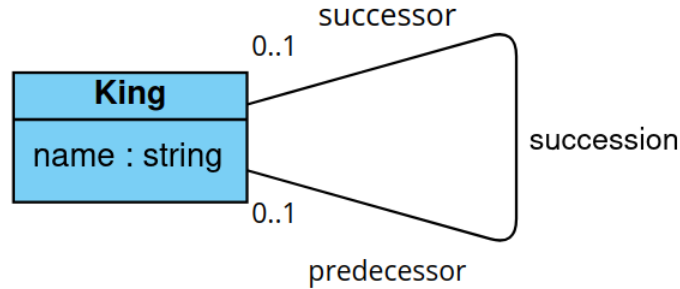
This is the correct version of the intentional scheme, a person can only have one home town while a city can be the home town of [0-hypotetically infinite] person

Important case

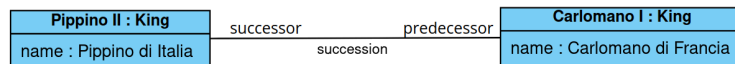
Suppose that we want to analyze the relationship between the kings of an almost destroyed kingdom. How we can order the kings with predecessor/successor relation?

- With an association from king to itself.

▮ In this case we can use additional identifier to specify the role of the hypothetical instances.

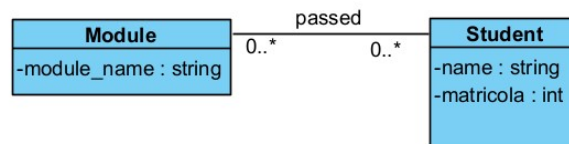


- A possible extensional level will be:



Associations with attributes

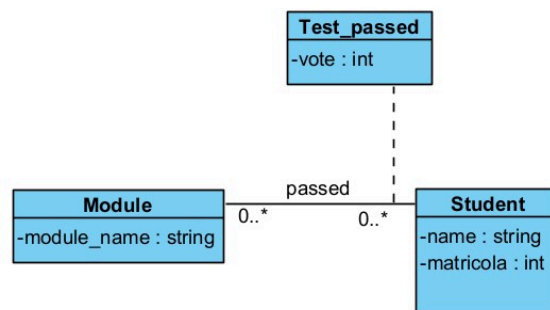
Suppose to have two classes: Module (that represent a module of an exam divided in modules) and Student.



They are in relationship thanks to an association: "passed".

Suppose that we want to know the vote that every student take in a specific exam, in which class I'll have to add the attribute vote?

In... 🙅 no one, because it is not an attribute of a specific exam and is not an attribute of a specific student, this attribute logically belongs to the association "passed".

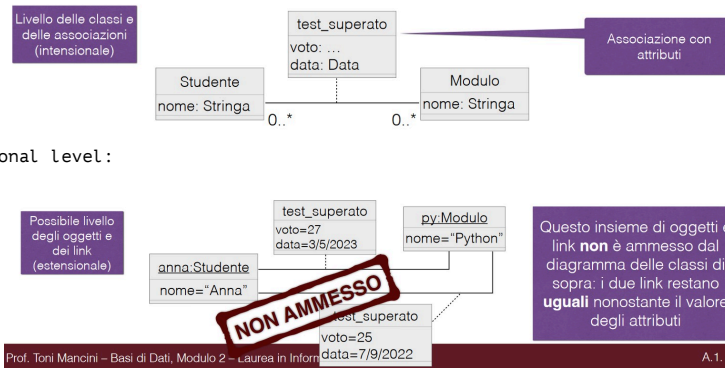


This type of association is called association with attributes.

The class used in the association with attributes is not a proper class but it can be in relationship with other classes through an association

The following case is not possible:

- Intencional level:



- (hypothetical) extensional level:

there cannot be two same links with attributes between two objects.

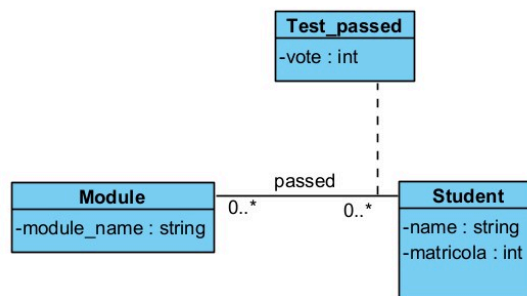
Data types

When a software engineer projects a project, this has to be more as possible disconnected to the technologies, to obtain this result, data types have to be more general as possible.

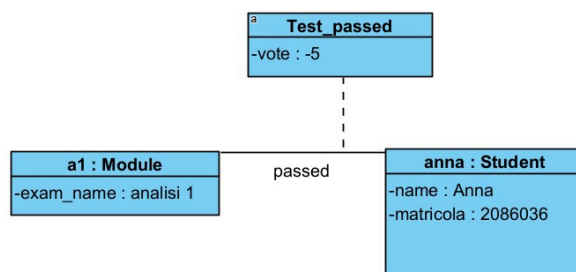
Data type as String, Date, DateTime, int, bool... is accepted because they are usable to all the existent technologies as Java, Python, C, C#, SQL...

Is given the following UML schema:

- Intensional level:



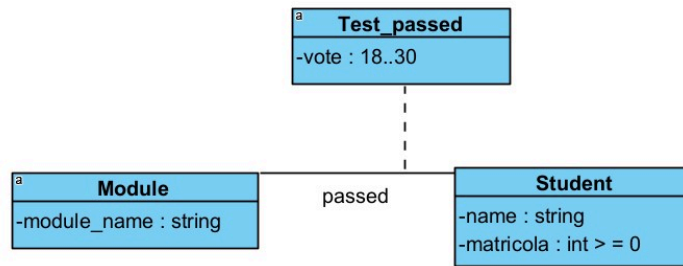
- Extensional level:



This is acceptable, but it doesn't resemble the reality (anna cannot pass the exam with vote = -5).

To avoid this errors we can limit the domain of the attribute "vote":

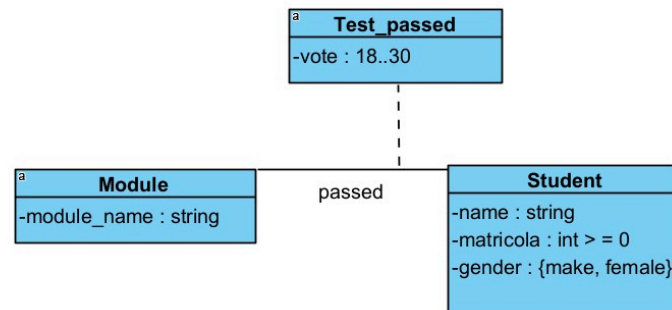
Intensional level:



Enumerative data types:

If the domain of an attribute is little and finite and we know all the possible values of the domain, for example: Genders, Continents, fingers (**eternal concepts that don't vary in time**) we can use an enumerative Type. Example:

Intentional level:



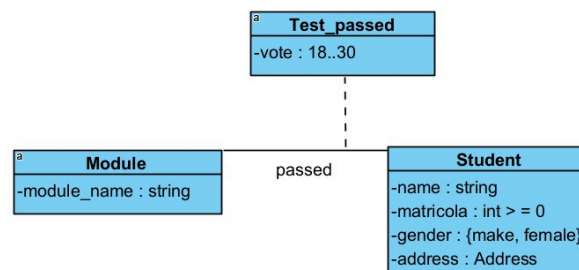
Compound data types

Compound data types are used to represent records. Example:

Address type: (civic number : $\text{int} \geq 0$, city : string, cap : $\text{int} \geq 0$)

Use $\text{int} \geq 0$ to represent civic number and cap is not a great idea..

Intentional level:

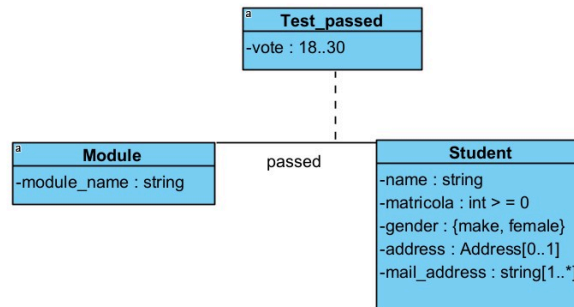


other Constraints

Multiplicity constraints

We can apply multiplicity constraints to class attributes and to attributes in class of association. Example:

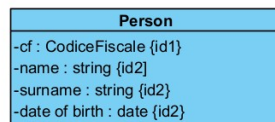
Intentional level:



In this case a student can have 0..1 address and 1..* mail address. If an attribute doesn't have a constraint, it is default set to [1..1].

Identification constraints of a class

Is given the following class:

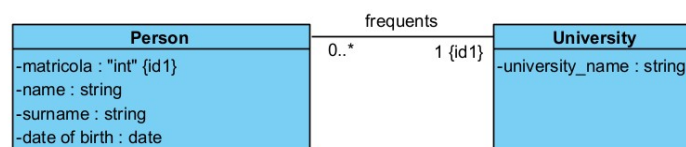


in the extensional level cannot exist two or more objects, instances of Person with the same cf or with same name, surname and date of birth.

{id2} is a single constraint on three attributes.

Identification constraint can involve associations between classes too.

example:



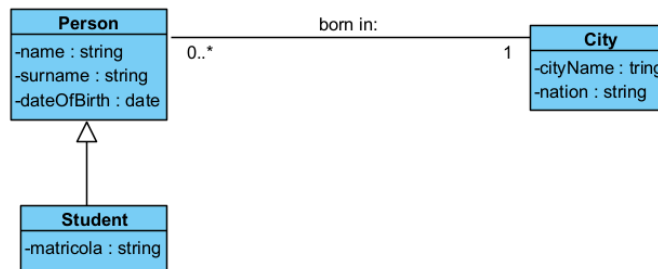
in this case, in extensional level cannot exist two or more objects, instances of Person, with the same matricola that frequent the same university.

{id1} is a single constraint on an attribute and an association.

Generalization

- "is a" relations. In UML it is possible to create classes "subset" of other classes.

example:

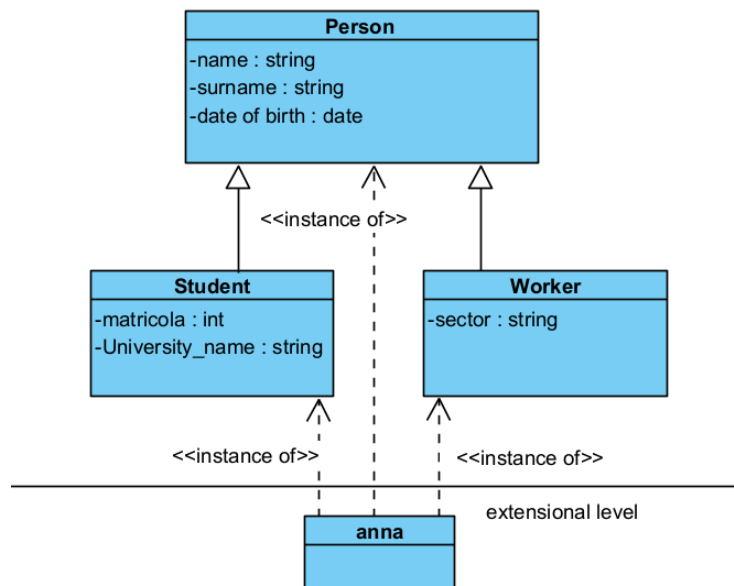


in this case the Student class is a subset of the Person class, Student class inherits all the attributes of Person and all the associations of Person.

student is always a person while person is not always a student. An instance of Person class is not always an instance of student class.

- A object in extensional level can be instance of more classes.

example:

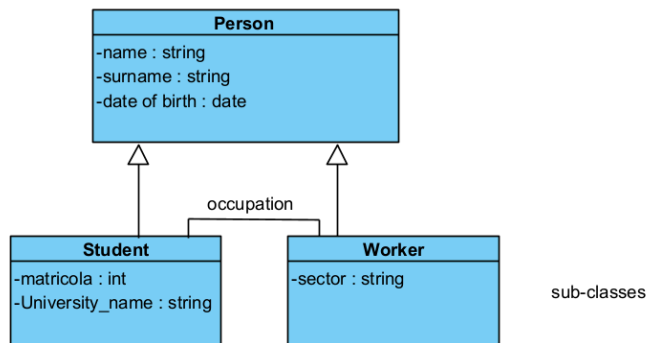


in fact, in this example anna is a student and a worker.

There is a more complex structure to better explain this concept, this is the generalization.

- Generalization allows to an object in extensional level to be instance of more sub-classes linked by a conceptual criteria.

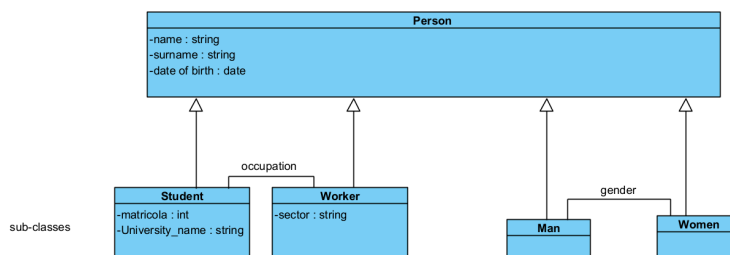
example:



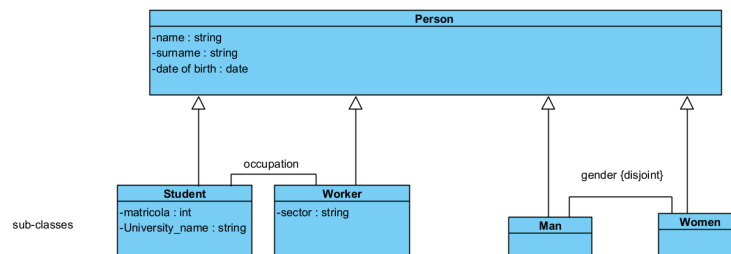
in this case occupation is the criteria. Anna can be a Student and a Worker, neither or both.

Generalization constraint

is given the following UML schema.

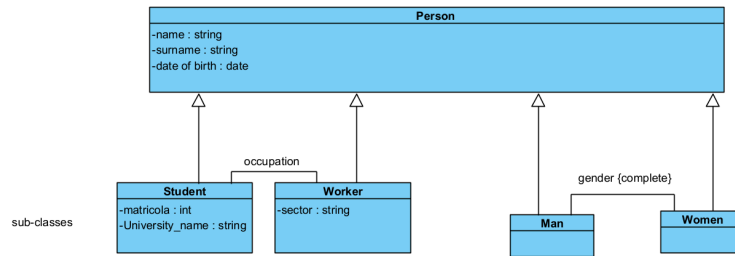


There is a problem! A person cannot be a man and a woman at the same time, we have to avoid this case. To do that there is in UML the generalization constraint "{disjoint}".

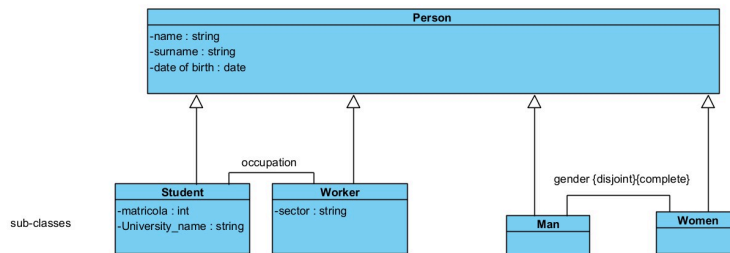


now the object in extensional level can be at least instance of one sub-class with gender criteria (Man or Woman).

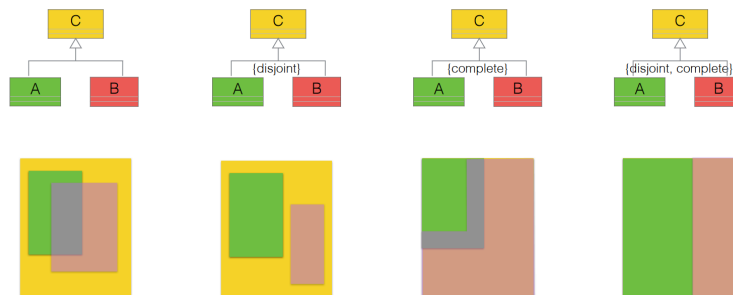
Another case avoid is the case where a person is not a man and is not a woman (I'm sorry, is only an example 🌈), To do that exist in UML the generalization constraint "{complete}".



Now, an instance of Person has to be also an instance of Man or an instance of woman or an instance of both.
We can merge this two constraints:



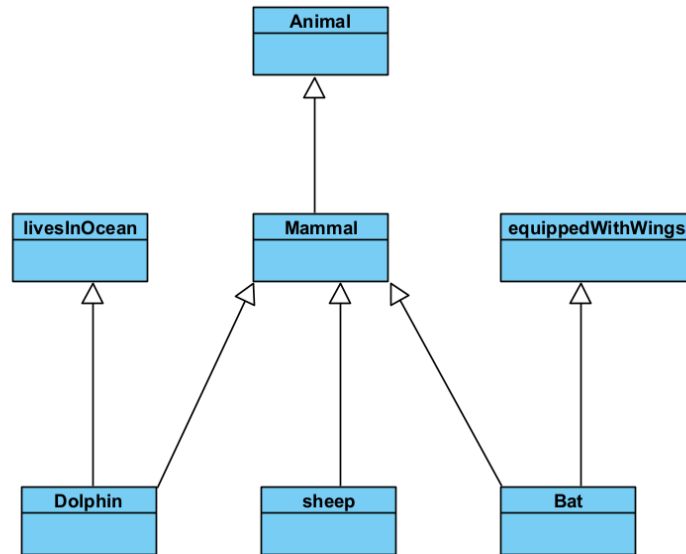
Now, an instance of Person class has to be also instance of Man or Woman.
Scheme:



- multiple inheritance

A class can be sub-class of multiple class, from classes the sub-class inherits all the attributes of the classes and all the associations.

It is a very dangerous operation, it can causes bug.



| In this case, multiple inheritance is wrongly used.

Operations

Until now we have seen only static properties of classes, but classes have also dynamic properties, these dynamic properties are called operations.

An operation can calculate a value using data or other operation in the class, can modify the state of an object or links that involve the object.

Syntax:

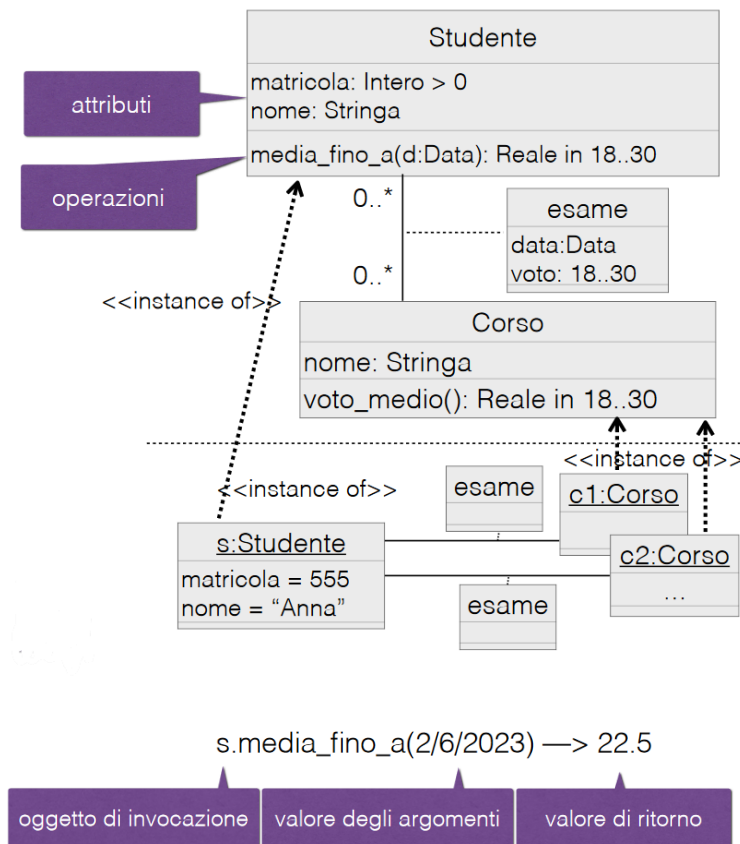
- `operation_name(arg) : return_type`

In “arg” we put arguments that the operation will use in the form: `argument_name : argument_type`. These arguments can be conceptual data type or a class of the diagram.

The return type is the type of data returned by the operation it can be a conceptual data type or a class of the diagram.

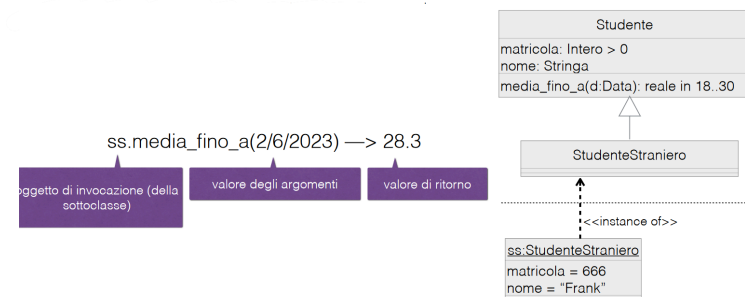
- An operation can be summoned only on an object of a class and it compare as an attribute(don't indicated in the extensional scheme).

example:



Operations still work on a subclass.

example:



Specification for class operations

For each operation there is a specification that describe what the operation do:

This specification consists in 2 parts:

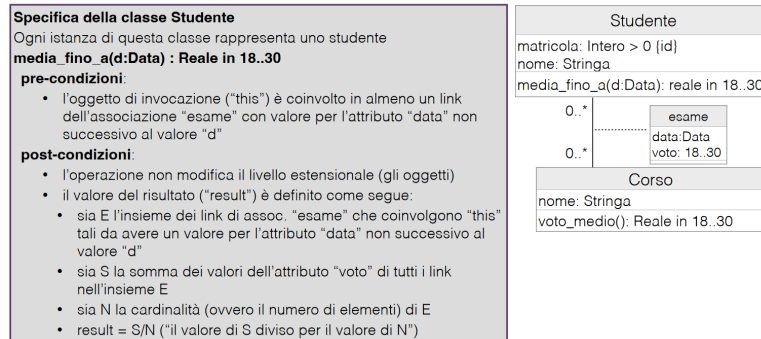
- pre-condition
- object summoned
- argument values
- other object
- post-condition

return type

definition of modify to apply to object in input, creation of new objects or links, elimination of objects, links.

In this specification we don't say how to do the operation but what is the scope of the operation.

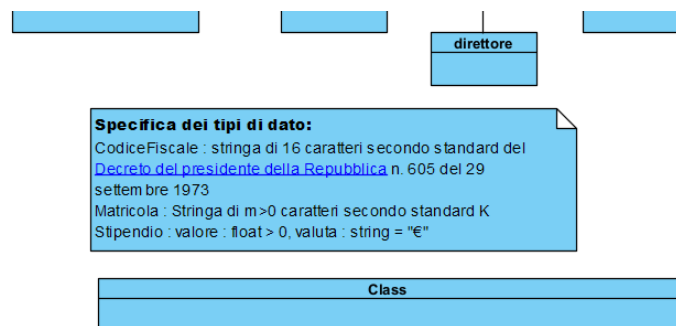
example:



Specification for new data_type

for each new data in the scheme we have to include in the documentation a specification where we define the new data type.

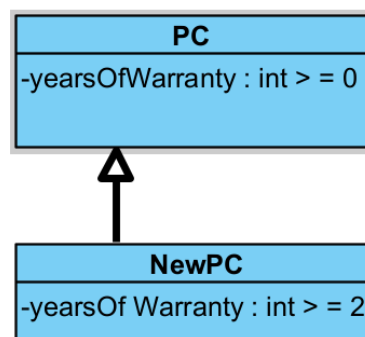
example:



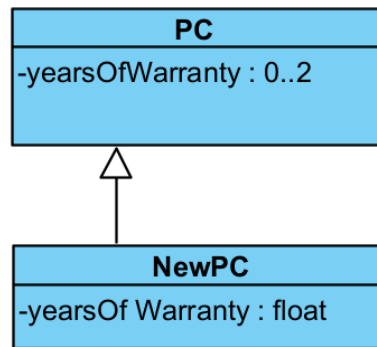
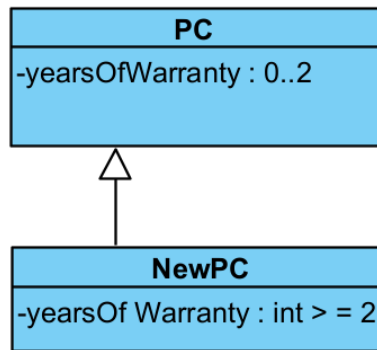
Attribute specialization

A sub-class can specialize properties inherited by a super-class (the reverse is not allowed)

example:



The following intensional schemes are not allowed.

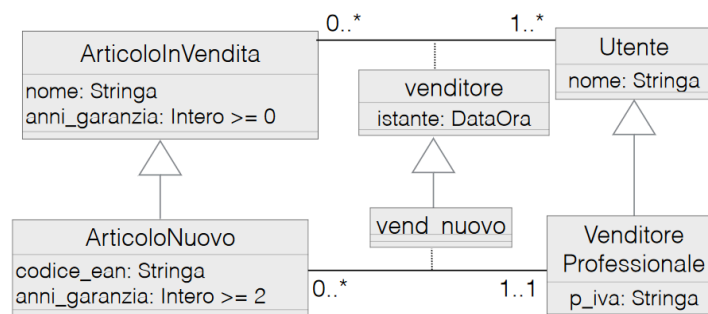


To be allowed the property has to be specialized.

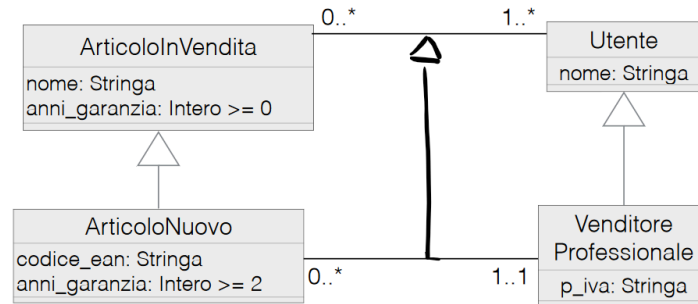
Association specialization

An association can be specialized

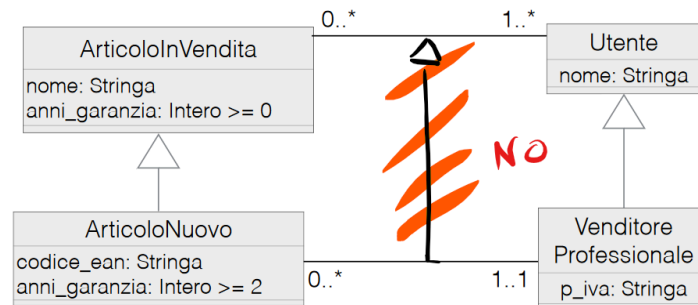
example:



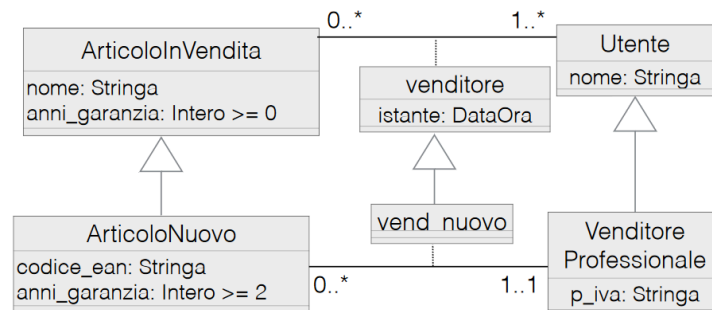
THIS PATTERN IS CALLED "NEEDLE IN THE HAYSTACK"



This is the meaning of the scheme but it is not allowed in UML

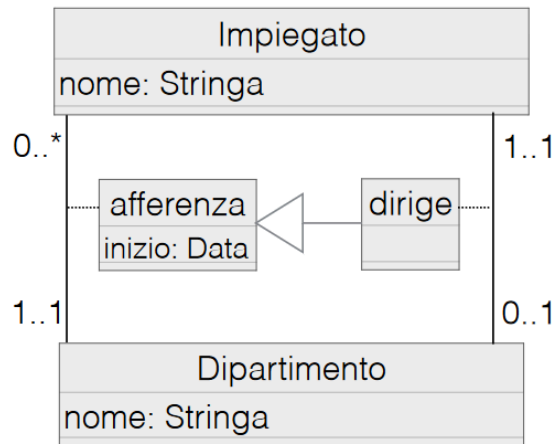


UML don't allowed generalization of associations...

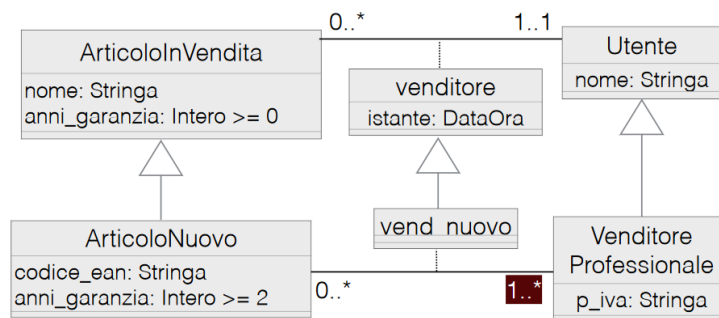


In this case a new article can has 1..* users that sell it but one of this has to be a professional vendor.

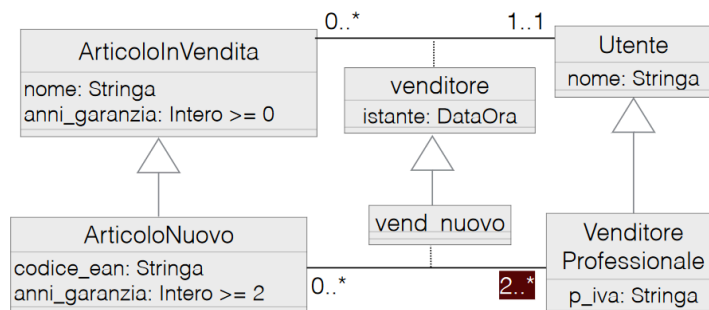
Other examples:



A lot of employers affers to a department but only one of these manage a department.



In this case we a new article can have an (generic) user that sells it and 1..* of this is can be professional vendor, one user has to be a professional vendor. in this case the article can be only one vendor and this vendor is a professional vendor.



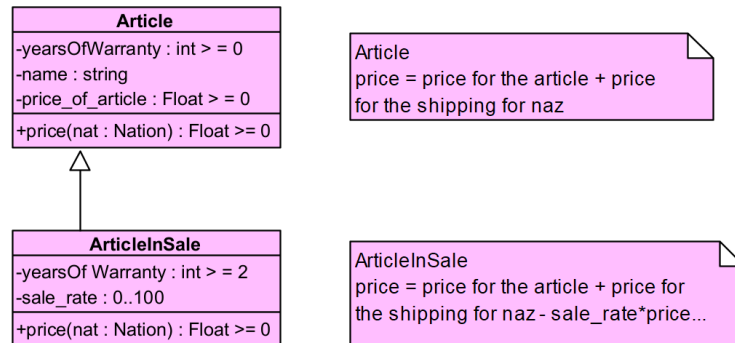
This case is impossible because we have 1 (generic) user that can sell a new article and, 2 of this has to be professional vendors.

examples in projects:

[GoPost1version.pdf](#)

Operation specialization

An operation can be specialized
example:



the operation of the subclass, the specialized operation, must have the same return type of the non-specialized operation and same number and type of arguments of the non-specialized operation.